

Exactly — **salt + SHA-256 is better than plain SHA-256, but it still isn't the right choice for password storage.** 👍

Let's separate the two jobs:

1. Salt solves one problem

```
password + unique random salt
      ↓
    hash function
```



The salt prevents identical passwords from producing identical hashes and defeats precomputed/rainbow-table attacks.

2. Argon2id solves a different problem

The big issue with SHA-256 is that it's **extremely fast**.

An attacker with a stolen database can do:

```
guess 1 → SHA-256
guess 2 → SHA-256
guess 3 → SHA-256
...
millions/billions of guesses
```



very quickly.

Argon2id is deliberately **slow and memory-intensive**:

```
password + unique salt
      ↓
    Argon2id
      ↓
    password hash
```



So an attacker's guessing process becomes much more expensive.

So is this safe?

Salt + SHA-256

```
password
+
random salt
```



```
↓  
SHA-256  
↓  
hash
```

Better than plaintext and better than unsalted SHA-256, but not recommended for password storage.

Salt + Argon2id

```
password  
+  
random salt  
↓  
Argon2id  
↓  
password hash
```



✅ This is the modern approach.

And importantly, **Argon2id handles the salt as part of its password-hashing process.** You generally don't need to manually do:

```
sha256(password + salt)
```

Instead, you use a password-hashing library:

```
const hash = await argon2.hash(password);
```

The library generates the salt, applies Argon2id, and produces the encoded hash containing the information needed for later verification.

Then login is simply:

```
const valid = await argon2.verify(  
  storedHash,  
  enteredPassword  
);
```

You don't manually:

```
entered password
+ retrieve salt
+ SHA-256
+ compare
```



The **Argon2** library does that work for you.

🧠 One-line mental model

Salt protects against precomputed attacks; Argon2id makes password guessing expensive.

That's why we use **Argon2id + a unique salt**, rather than trying to build our own **salt + SHA-256** password system. 🗝️



okay

😊👍 Perfect.

Now the password part is locked in:

```
Password
↓
Argon2id + unique salt
↓
Stored password hash
```



And during login:

```
Entered password
↓
Argon2id verification
↓
Matches stored hash?
├─ Yes → Authentication succeeds ✅
└─ No  → Login fails ❌
```



Next, we'll jump back into **7.4 — actual Next.js login code** and see exactly how this connects to **Server Actions** → **DB** → **session** → `cookies()` → **redirect**. 🔥



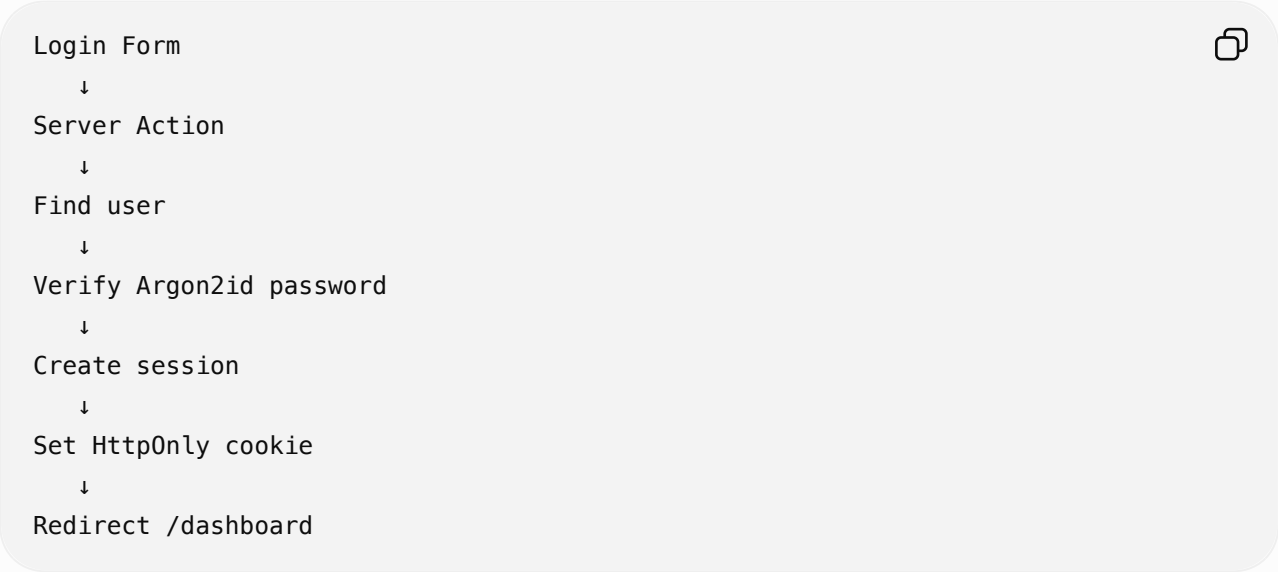
okay

🔥 Let's continue.

7.4 — Login Flow in Actual Next.js

Now we're going to turn the theory into code.

We'll build this flow:



We'll deliberately **not jump into a full auth library yet**. I want you to understand what the framework is doing first.

7.4.1 — Our imaginary database

Let's assume we have:

users		
id	email	passwordHash
42	<u>gopi@example.com ↗</u>	\$argon2id\$...

sessions		
id	userId	expiresAt
abc123...	42	future date

The relationship is:



7.4.2 — Login page

Our page could look like:

`</>` TypeScript



```
export default function LoginPage() {
  return (
    <form action={login}>
      <input
        name="email"
        type="email"
        placeholder="Email"
      />

      <input
        name="password"
        type="password"
        placeholder="Password"
      />

      <button type="submit">
        Login
      </button>
    </form>
  );
}
```

And `login` is a Server Action:

`</>` TypeScript



```
"use server";

export async function login(formData: FormData) {
  // login logic
}
```

Notice something important

We don't have:

```
</> TypeScript
onClick={() => fetch("/api/login")}
```



We're using:

```
</> TypeScript
<form action={login}>
```



Next.js knows:

This form is invoking a Server Action.

The implementation of `login()` executes on the server.

7.4.3 — Get the submitted values

Inside the Server Action:

```
</> TypeScript
"use server";

export async function login(formData: FormData) {
  const email = formData.get("email");
  const password = formData.get("password");
}
```



Suppose the user entered:

```
email = gopi@example.com
password = hello123
```



Then:

```
</> TypeScript
email
// "gopi@example.com"

password
// "hello123"
```



These values are now available **on the server**.

7.4.4 — Find the user

We query the DB:

```
</> TypeScript

const user = await db.user.findUnique({
  where: {
    email: email as string,
  },
});
```

If found:

```
</> TypeScript

user = {
  id: 42,
  email: "gopi@example.com",
  passwordHash: "$argon2id$..."
};
```

If not:

```
</> TypeScript

user = null;
```

We shouldn't continue if there is no user.

```
</> TypeScript

if (!user) {
  return {
    error: "Invalid email or password",
  };
}
```

Notice we're using a **generic error** rather than saying:

```
"Email doesn't exist"
```

This helps avoid revealing which email addresses have accounts.

7.4.5 — Verify the password

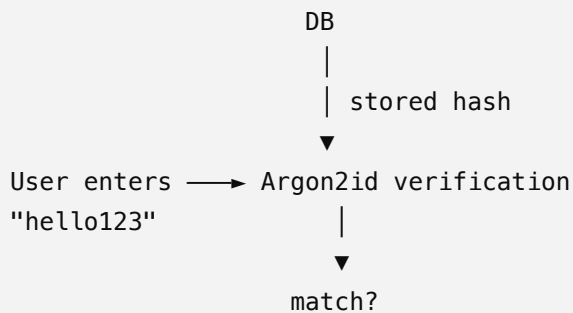
Now:

</> TypeScript

```
const validPassword = await argon2.verify(  
  user.passwordHash,  
  password as string  
);
```



Conceptually:



If false:

</> TypeScript

```
if (!validPassword) {  
  return {  
    error: "Invalid email or password",  
  };  
}
```



If true:

Authentication successful



At this point we've answered:

Who is this user?

User:

id = 42



7.4.6 — Create the session

Now we generate a random session ID.

Conceptually:

`</> TypeScript`

```
const sessionId = crypto.randomUUID();
```



You could get something like:

8f5d9e3a-...



Then save:

`</> TypeScript`

```
await db.session.create({
  data: {
    id: sessionId,
    userId: user.id,
    expiresAt: /* future date */,
  },
});
```



Database:

sessions



id	userId
8f5d9e3a-...	42

Now the server has established:

session ID 8f5d9e3a...

↓

User 42



7.4.7 — Set the cookie 🍪

Now comes the Next.js-specific part.

We can use:

</> TypeScript



```
import { cookies } from "next/headers";
```

Then:

</> TypeScript



```
const cookieStore = await cookies();

cookieStore.set(
  "sessionId",
  sessionId,
  {
    httpOnly: true,
    secure: true,
    sameSite: "lax",
    expires: expiresAt,
    path: "/",
  }
);
```

Let's understand each one.

httpOnly

</> TypeScript



```
httpOnly: true
```

Means browser JavaScript cannot access the cookie through:

</> JavaScript



```
document.cookie
```

That's important for reducing exposure during certain XSS attacks.

secure

</> TypeScript



```
secure: true
```

Means the browser should only send the cookie over HTTPS.

For production:

```
https://myapp.com
```



not:

```
http://myapp.com
```



sameSite

```
</> TypeScript
```



```
sameSite: "lax"
```

Controls when the browser sends the cookie in cross-site situations and helps mitigate CSRF.

We'll go much deeper into this when we get to the security portion.

path

```
</> TypeScript
```



```
path: "/"
```

Means the cookie applies across the site.

So:

```
/dashboard  
/profile  
/settings
```



can all receive it.

7.4.8 — Redirect

Now that authentication is complete:

```
</> TypeScript
```



```
redirect("/dashboard");
```

So:

```
Login successful
```

```
↓
```

```
session created
```

```
↓
```

```
cookie set
```

```
↓
```

```
redirect
```

```
↓
```

```
/dashboard
```



7.4.9 — What happens during /dashboard ?

This is where the whole thing becomes beautiful. 😊

Browser requests:

```
GET /dashboard
```



And automatically includes:

```
</> http
```



```
Cookie: sessionId=8f5d9e3a...
```

The server reads it:

```
</> TypeScript
```



```
const cookieStore = await cookies();
```

```
const sessionId = cookieStore.get("sessionId")?.value;
```

Now:

```
sessionId
```

```
↓
```

```
8f5d9e3a...
```



Server queries:

</> TypeScript



```
const session = await db.session.findUnique({
  where: {
    id: sessionId,
  },
});
```

DB says:

8f5d9e3a... → userId 42



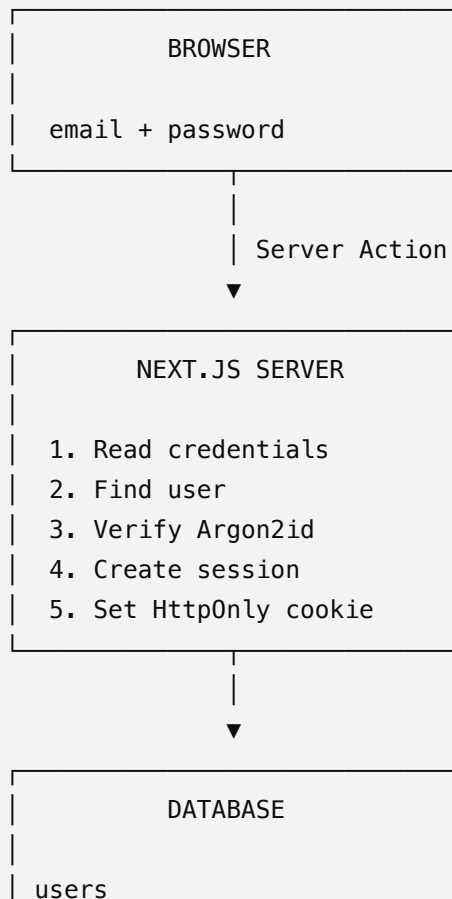
Now the server knows:

Current user = 42

 That's authentication on subsequent requests.

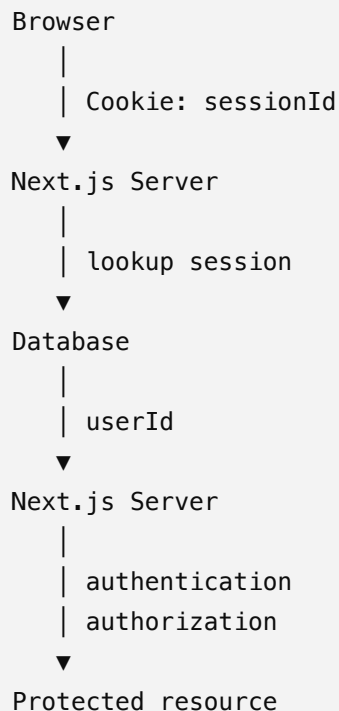
The complete Next.js mental model

Keep this picture:



sessions

Then every later request:



🧠 One thing I really want you to notice

The **password** is involved only during login.

After successful login:

password
✗

isn't repeatedly sent around.

Instead:

sessionId
✓

is what identifies the authenticated session.

That's why the architecture becomes:

LOGIN:


```
email + password
  ↓
authentication
  ↓
session
```

AFTER LOGIN:

```
session cookie
  ↓
identify session
  ↓
identify user
  ↓
authorization
```

This distinction is **huge** for understanding authentication.

Next piece: `getCurrentUser()`

Rather than writing this every time:

`</>` TypeScript



```
const cookieStore = await cookies();
const sessionId = cookieStore.get("sessionId");
const session = await db.session.findUnique(...);
```

we normally centralize it into something like:

`</>` TypeScript



```
const user = await getCurrentUser();
```

Then you'll see how this gets used inside **Server Components**, layouts, Server Actions, and Route Handlers — which leads directly into **7.5 Server Components + Auth**. 🔥





Awesome 😊 Let's move to the **next important Auth.js concept: how the session is actually used to protect pages and APIs.**

Lesson 7.x — Protecting Routes with Auth.js

We've now understood:

Google

↓

OAuth/OIDC

↓



```
Auth.js callback
  ↓
User/Account
  ↓
Session
  ↓
Cookie
```

Now the natural question is:

Once the user is logged in, how do I actually stop an unauthenticated user from accessing `/dashboard` or `/api/orders` ?

This is where `auth()` becomes really important.

1. Protecting a Server Component

Suppose we have:

```
app/
├─ login/
│   └─ page.tsx
└─ dashboard/
    └─ page.tsx
```

Our dashboard should only be accessible to logged-in users.

We can write:

```
<\/> TypeScript

// app/dashboard/page.tsx

import { auth } from "@auth";
import { redirect } from "next/navigation";

export default async function DashboardPage() {
  const session = await auth();

  if (!session) {
    redirect("/login");
  }

  return (
    <div>
      <h1>Welcome {session.user?.name}<\/h1>
      <p>{session.user?.email}<\/p>
    <\/div>
  );
}
```

```
    </div>
  );
}
```

Now let's understand **exactly what happens**.

2. User requests `/dashboard`

Browser:

```
</> http

GET /dashboard
Cookie: authjs.session-token=ABC123
```



Next.js receives this request.

Because `dashboard/page.tsx` is a **Server Component**, it executes on the server.

Then:

```
</> TypeScript

const session = await auth();
```



runs.

3. What does `auth()` do?

Remember where it came from:

```
</> TypeScript

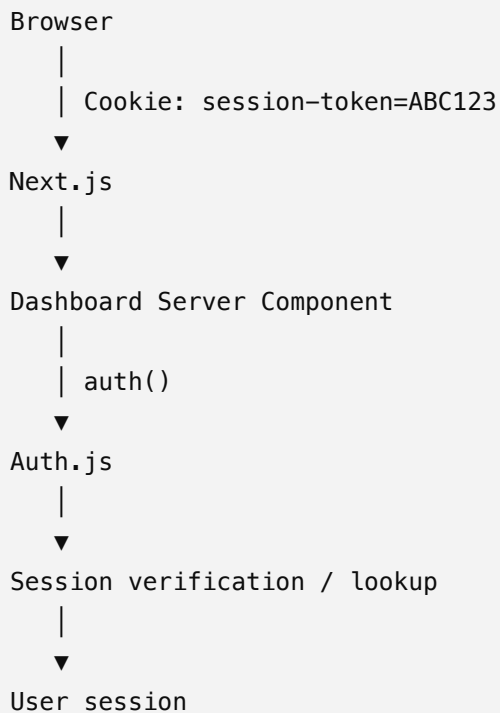
// auth.ts

export const {
  handlers,
  auth,
  signIn,
  signOut
} = NextAuth({...});
```



So `auth()` is an Auth.js server-side function.

Conceptually:



If the session is valid:

</> TypeScript

```
session = {
  user: {
    name: "Gopi",
    email: "gopi@example.com"
  }
}
```

Then the page renders.

4. What if the user isn't logged in?

Suppose there is no valid session.

Then:

</> TypeScript

```
const session = await auth();
```

returns something equivalent to:

</> TypeScript

```
null
```

So:

```
</> TypeScript

if (!session) {
  redirect("/login");
}
```



executes.

Next.js sends a redirect response.

Conceptually:

```
</> http

307 Temporary Redirect
Location: /login
```



The browser then requests:

```
</> http

GET /login
```



So:

```
User → /dashboard
      ↓
    auth()
      ↓
  no session
      ↓
redirect("/login")
      ↓
    /login
```



5. Why isn't this authorization?

This is an important distinction.

This code checks:

```
</> TypeScript
```



```
if (!session)
```

That is **authentication**.

We're asking:

"Is this person logged in?"

But suppose we have:

Admin Dashboard



and only admins can access it.

Then we need **authorization**.

For example:

```
</> TypeScript
```



```
const session = await auth();

if (!session) {
  redirect("/login");
}

if (session.user.role !== "ADMIN") {
  redirect("/forbidden");
}
```

Now we're asking two different questions:

Authentication:

"Who are you / are you logged in?"



Authorization:

"Are you allowed to do this?"

6. But there's an important security rule

Never rely only on hiding UI.

For example:

```
</> TypeScript
```



```
{session?.user.role === "ADMIN" && (  
  <button>Delete User</button>  
)}
```

This is useful for UX.

But it is **not security**.

A malicious user can manually send a request to your server.

For example:

```
</> http  
  
POST /api/users/delete
```



Therefore your server-side action/API must check authorization too.

7. Protecting a Route Handler

Suppose we have:

```
app/api/orders/route.ts
```



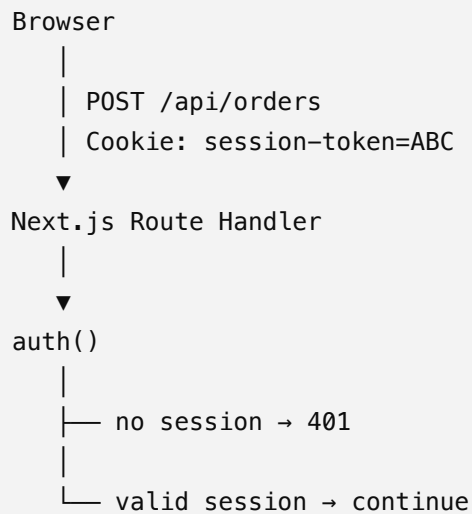
We can do:

```
</> TypeScript  
  
import { auth } from "@auth";  
import { NextResponse } from "next/server";  
  
export async function POST(request: Request) {  
  const session = await auth();  
  
  if (!session) {  
    return NextResponse.json(  
      { error: "Unauthorized" },  
      { status: 401 }  
    );  
  }  
  
  // User is authenticated  
  // Continue processing order...  
  
  return NextResponse.json({  
    success: true,  
  });  
}
```



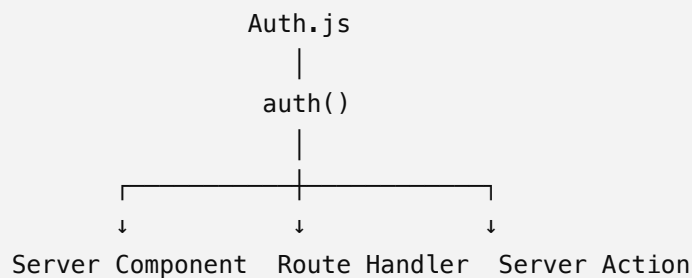
```
});  
}
```

Now:



8. This is why `auth()` is so useful

You can use the same server-side authentication context in multiple places:



For example:

Server Component

</> TypeScript

```
const session = await auth();
```

Route Handler

</> TypeScript

```
const session = await auth();
```

Server Action

⟨⟩ TypeScript



```
const session = await auth();
```

The exact mechanics differ depending on the execution context, but the idea is:

Your server-side code asks Auth.js to resolve the current authenticated user/session.

9. Server Action example

Suppose:

⟨⟩ TypeScript



```
"use server";

import { auth } from "@auth";

export async function deleteAccount() {
  const session = await auth();

  if (!session) {
    throw new Error("Unauthorized");
  }

  // Delete account...
}
```

Even if your UI hides the "Delete Account" button from logged-out users, the Server Action still checks authentication.

That's the security boundary.

10. Interview answer 🎯

If an interviewer asks:

"How would you protect a page in Next.js using Auth.js?"

A strong answer would be:

"For server-rendered pages, I can call Auth.js's server-side `auth()` function to resolve the current session. If there's no valid session, I redirect the user to the login page. For protected mutations or APIs, I perform the authentication and authorization checks again inside the Server Action or Route Handler rather than relying on client-side UI checks."

And if they ask:

"Is hiding an admin button authorization?"

Say:

"No. That's only a UI concern. Authorization must be enforced on the server at the resource or mutation boundary."

One thing I want you to notice

We now have **three different places** where Auth.js can participate:

1. LOGIN

```
signIn()  
  ↓  
OAuth  
  ↓  
callback  
  ↓  
session created
```



2. READING SESSION

```
auth()  
  ↓  
"Who is currently logged in?"
```

3. PROTECTING ACTIONS

```
auth()  
  ↓  
authenticate  
  ↓  
authorize  
  ↓  
perform operation
```

That distinction is **very interview-important**.

Next after this

The natural next piece is **Auth.js callbacks** — specifically:

`</> TypeScript`



```
callbacks: {  
  async jwt() {},  
  async session() {},  
  async signIn() {},  
  async redirect() {}  
}
```

Because you've now seen `session` being created and consumed, the next question is:

"How do I customize what gets stored in the JWT/session — for example, how do I put a user's `role` or `id` into the session?"

That's where the `jwt()` and `session()` callbacks finally make sense. 🔥



how many more are there

Not too many 😊 We're actually pretty close to wrapping the **Authentication & Authorization** section.

Based on what we've covered so far, I'd structure the remaining part like this:

Remaining Auth.js / Auth & Authorization lessons

#	Topic	What you'll learn
7.6	Route Protection	<code>auth()</code> , protecting pages, APIs, Server Actions
7.7	Auth.js Callbacks	<code>signIn</code> , <code>jwt</code> , <code>session</code> , <code>redirect</code> callbacks
7.8	Session Customization	Put <code>user.id</code> , <code>role</code> , etc. into the session
7.9	Authorization / RBAC	Roles, permissions, admin/user access
7.10	Session Strategies	Database sessions vs JWT sessions

#	Topic	What you'll learn
7.11	 CSRF + SameSite	Why state-changing requests need protection
7.12	 Middleware / Proxy Auth	Protecting groups of routes before rendering
7.13	 Logout & Session Expiry	Sign-out, expiration, invalid/expired sessions
7.14	 Complete Auth Architecture	Put Browser → Next.js → Auth.js → OAuth → DB → Session together
7.15	 Auth Interview Questions	Real interview scenarios + architecture questions

So **about 10 lessons remain**, depending on how deeply we split things.

But some of these are short. The **big ones** are:

1. Callbacks
2. Session customization
3. Authorization/RBAC
4. JWT vs DB sessions
5. CSRF/SameSite
6. Middleware/Proxy

After that, I'd consider the Auth section **complete** and move you to the next major Next.js/interview area. 🚀

And I wouldn't rush through these—especially callbacks and authorization. Given the way you've been learning, I'd rather make sure you can explain the **actual request flow** than just memorize Auth.js APIs.



so after this authentication and authorization are done?

Yes. 👍 After we finish those remaining Auth.js topics, we can consider the **Authentication & Authorization section complete**.

More precisely, your **Next.js Auth & Authorization roadmap** will cover:

AUTHENTICATION



- ✓ Authentication vs Authorization
- ✓ Auth architecture
- ✓ Cookies
- ✓ Sessions
- ✓ JWT
- ✓ Password hashing
- ✓ Login flow
- ✓ Server Components + auth
- ✓ Client Components + auth
- ✓ Auth.js
- ✓ OAuth 2.0
- ✓ OpenID Connect
- ✓ Google authentication
- ✓ Authorization code + callback
- ✓ Client ID / Client Secret
- ✓ Token endpoint
- ✓ Auth.js adapters
- ✓ ORM + database persistence
- ✓ User / Account / Session
- ✓ Database sessions vs JWT sessions
- ✓ Route protection
- ✓ Auth.js callbacks
- ✓ Session customization

AUTHORIZATION

- ✓ Authentication vs authorization
- ✓ Protecting Server Components
- ✓ Protecting Route Handlers
- ✓ Protecting Server Actions
- ✓ Roles / RBAC
- ✓ Permissions
- ✓ Server-side authorization
- ✓ Client UI vs actual security boundary

SECURITY

- ✓ HttpOnly / Secure / SameSite
- ✓ CSRF
- ✓ Session expiry
- ✓ Logout
- ✓ Middleware / Proxy protection
- ✓ Common auth vulnerabilities

Then we'll do **one complete end-to-end architecture walkthrough**:



```
graph TD
    Browser --> Login
    Login --> Authjs[Auth.js]
    Authjs --> Google[Google OAuth/OIDC]
    Google --> Callback
    Callback --> GoogleToken[Google token exchange]
    GoogleToken --> User[User + Account]
    User --> Adapter
    Adapter --> Database
    Database --> Session
    Session --> Cookie
    Cookie --> ProtectedRequest[Protected request]
    ProtectedRequest --> auth[auth()]
    auth --> Authentication
    Authentication --> Authorization
    Authorization --> ProtectedResource[Protected resource]
```

That final walkthrough is important because you've been asking the right kind of **"but who actually calls what?"** questions. 😊 It will tie all the individual pieces together.

So yes: once we finish these, you can confidently mark *Next.js Authentication & Authorization* as DONE. 🗝️✅



ChatGPT can make mistakes. Check important info.